SIMATS Online Lab

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=389

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

H Harshavardhan

CO2_AT2_Scenario Based Program Writing

← Back

QUESTIONS

- Q1 Intelligent Customer Feedback Analysis 25 marks
- Q2 Comprehensive Password Strength Checker 25 marks
- Q3 Structured Server Log Error Analyser 25 marks
- Q4 Email Domain Frequency Analyser 25 marks

4/4 answered

Q1 Intelligent Customer Feedback Analysis

25 marks

Context: An e-commerce platform collects large volumes of customer feedback in unstructured textual form. The feedback often contains inconsistent capitalization, punctuation, and unnecessary spacing. The system must preprocess the text by normalizing case and removing unwanted characters. It should then identify specific keywords that indicate positive or negative sentiment. The program must count the occurrences of such keywords efficiently. Based on the counts, the system should classify the feedback into categories. The classification logic must be implemented using appropriate control flow constructs. The solution should be modularized using functions for preprocessing and analysis. It should also ensure scalability for handling multiple feedback entries.

Task:

Design a Python program to implement this intelligent feedback analysis system.

Key Skills to Demonstrate:

- String normalization: lower(), strip(), replace()
- Keyword counting using loops and conditionals
- Sentiment classification using if-elif-else
- Modular design with functions
- Handling multiple feedback entries using loops

Your Code

```
pos=["good","great","happy","excellent","love","fast"]
neg=["bad","poor","worst","hate","slow","delay"]
def analyse(fb):
    fb=fb.lower().strip()
    fb=fb.replace(".","")
    fb=fb.replace(",","")
    fb=fb.replace("!","")
    fb=fb.replace("?","")
    words=fb.split()
    poss=0
    negg=0
```

Input

```
2
the product is good
the product is great
```

Output

```
positive keywords: 1
negative keywords: 0
sentiment:positive
positive keywords: 1
negative keywords: 0
```

Analyser
25 marks

4/4 answered

- Modular design with functions
- Handling multiple feedback entries using loops

Your Code

```
pos=["good","great","happy","excellent","love","fast"]
neg=["bad","poor","worst","hate","slow","delay"]
def analyse(fb):
    fb=fb.lower().strip()
    fb=fb.replace(".","")
    fb=fb.replace(",","")
    fb=fb.replace("!","")
    fb=fb.replace("?","")
    words=fb.split()
    poss=0
    negg=0
    for word in words:
        if word in pos:
            poss+=1
        elif word in neg:
            negg+=1
    print("positive keywords:",poss)
    print("negative keywords:",negg)
    if poss>negg:
        print("sentiment:positive")
    elif negg>poss:
        print("sentiment:negative")
    else:
        print("sentiment:neutral")
n=int(input())
for i in range(n):
    fb=input()
    analyse(fb)
```

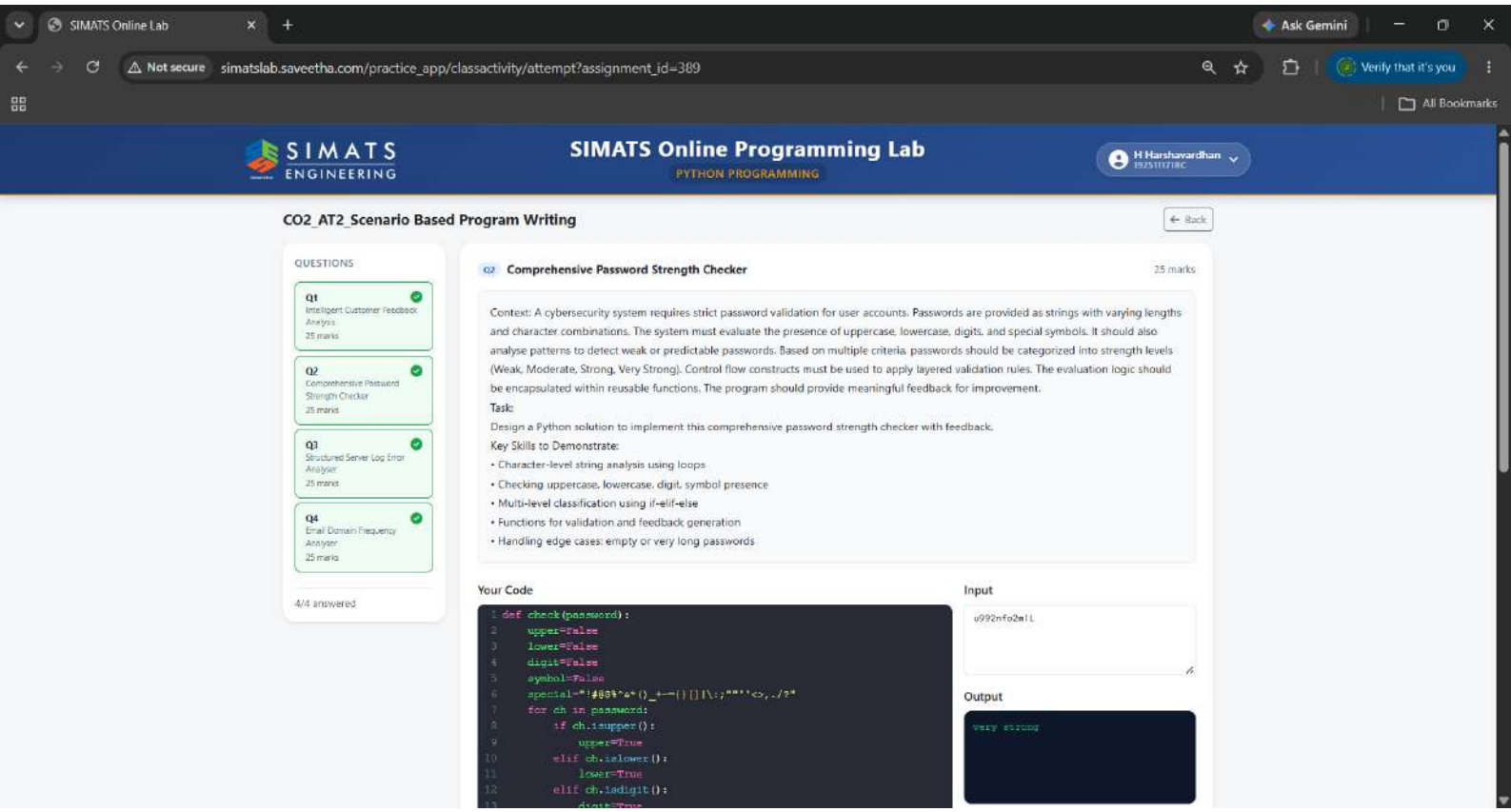
Submitted

Input

```
2
the product is good
the product is great
```

Output

```
positive keywords: 1
negative keywords: 0
sentiment:positive
positive keywords: 1
negative keywords: 0
```


SIMATS Online Lab
Ask Gemini
Not secure
simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=389
Verify that it's you
All Bookmarks
SIMATS ENGINEERING
SIMATS Online Programming Lab
PYTHON PROGRAMMING
H Harshavardhan
CO2_AT2_Scenario Based Program Writing
Back
QUESTIONS
Q1
Intelligent Customer Feedback Analysis
25 marks
Q2
Comprehensive Password Strength Checker
25 marks
Q3
Structured Server Log Error Analyser
25 marks
Q4
Email Domain Frequency Analyser
25 marks
4/4 answered
Q2 Comprehensive Password Strength Checker
25 marks
Context: A cybersecurity system requires strict password validation for user accounts. Passwords are provided as strings with varying lengths and character combinations. The system must evaluate the presence of uppercase, lowercase, digits, and special symbols. It should also analyse patterns to detect weak or predictable passwords. Based on multiple criteria, passwords should be categorized into strength levels (Weak, Moderate, Strong, Very Strong). Control flow constructs must be used to apply layered validation rules. The evaluation logic should be encapsulated within reusable functions. The program should provide meaningful feedback for improvement.
Task:
Design a Python solution to implement this comprehensive password strength checker with feedback.
Key Skills to Demonstrate:
• Character-level string analysis using loops
• Checking uppercase, lowercase, digit, symbol presence
• Multi-level classification using if-elif-else
• Functions for validation and feedback generation
• Handling edge cases: empty or very long passwords
Your Code
def check(password):
upper=False
lower=False
digit=False
symbol=False
special="!@$%^&*()_+-={}[]|\:;"'<>,./?"
for ch in password:
if ch.isupper():
upper=True
elif ch.islower():
lower=True
elif ch.isdigit():
Input
o992nfo2mlL
Output
very strong

SIMATS Online Lab

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=389

Ask Gemini | Verify that it's you | All Bookmarks

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

H Harshavardhan 192511127IRC

4/4 answered

Your Code

```python
def check(password):
    upper=False
    lower=False
    digit=False
    symbol=False
    special="!@#$%^&*()_+-=[]{}|\:;"'<>,./?"
    for ch in password:
        if ch.isupper():
            upper=True
        elif ch.islower():
            lower=True
        elif ch.isdigit():
            digit=True
        elif ch in special:
            symbol =True
    score=0
    if len(password)>=8:
        score+=1
    if upper:
        score+=1
    if lower:
        score+=1
    if digit:
        score+=1
    if symbol:
        score+=1
    if score<=2:
        print("weak")
    if score==3:
        print("moderate")
    if score==4:
        print("strong")
    else:
        print("very strong")
    if not upper:
        print("add uppercase letter")
    if not lower:
```

Input

```
u992nfo2mIL
```

Output

```
very strong
```

```
    score=0
    if len(password)>=8:
        score+=1
    if upper:
        score+=1
    if lower:
        score+=1
    if digit:
        score+=1
    if symbol:
        score+=1
    if score<=2:
        print("weak")
    if score==3:
        print("moderate")
    if score==4:
        print("strong")
    else:
        print("very strong")
    if not upper:
        print("add uppercase letter")
    if not lower:
        print("add lowercase letter")
    if not digit:
        print("add digit")
    if not symbol:
        print("add specail symbol")
    if len(password)<8:
        print("password should be at least 8 character")
password=input()
check(password)
```

Submitted

SIMATS Online Lab

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=389

Ask Gemini | Verify that it's you | All Bookmarks

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

H Harshavardhan 192511177IRC

CO2_AT2_Scenario Based Program Writing

← Back

QUESTIONS

- Q1 Intelligent Customer Feedback Analysis 25 marks
- Q2 Comprehensive Password Strength Checker 25 marks
- Q3 Structured Server Log Error Analyser 25 marks
- Q4 Email Domain Frequency Analyser 25 marks

4/4 answered

Q3 Structured Server Log Error Analyser

25 marks

Context: A server continuously generates log files containing various system messages. These logs include informational, warning, and error messages in text format. The system must extract only error-related entries for analysis. String searching techniques should be used to identify relevant lines. The program should count occurrences of different types of errors. It must also remove duplicate error messages for concise reporting. Control flow should manage filtering and classification of log entries. Functions must be used to modularize extraction and summarization tasks.

Task:

Develop a Python program to perform structured log error analysis and reporting.

Key Skills to Demonstrate:

- String searching: in, find(), startswith()
- Filtering using for loop and conditionals
- Duplicate removal logic
- Error counting and categorization
- Functions for extraction, filtering, and summarization

Your Code

```
def analyze(logs):
    unique=[]
    total=0
    for line in logs:
        if "error" in line.lower():
            total+=1
            if line not in unique:
                unique.append(line)
    print("total errors:",total)
    print("total unique errors:",len(unique))
    for e in unique:
        c=0
        for line in logs:
```

Input

```
5
info server started
error database failed
waring memory low
```

Output

```
total errors: 3
total unique errors: 2
error database failed - 2
error disk full - 1
```

SIMATS Online Lab

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=389

Verify that it's you

All Bookmarks

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

H Harshavardhan
192511177IRC

Q3
Structured Server Log Error Analyser
25 marks

Q4
Email Domain Frequency Analyser
25 marks

4/4 answered

Key Skills to Demonstrate:

- String searching: in, find(), startswith()
- Filtering using for loop and conditionals
- Duplicate removal logic
- Error counting and categorization
- Functions for extraction, filtering, and summarization

Your Code

```
def analyze(logs):
    unique=[]
    total=0
    for line in logs:
        if "error" in line.lower():
            total+=1
            if line not in unique:
                unique.append(line)
    print("total errors:",total)
    print("total unique errors:",len(unique))
    for e in unique:
        c=0
        for line in logs:
            if line==e:
                c+=1
        print(e,"-",c)
n=int(input())
logs=[]
for i in range(n):
    logs.append(input())
analyze(logs)
```

Input

```
5
info server started
error database failed
waring memory low
```

Output

```
total errors: 3
total unique errors: 2
error database failed - 2
error disk full - 1
```

Submitted

SIMATS Online Lab

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=389

Ask Gemini | Verify that it's you | All Bookmarks

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

H Harshavardhan 192511177IRC

QUESTIONS

- Q1 Intelligent Customer Feedback Analysis 25 marks
- Q2 Comprehensive Password Strength Checker 25 marks
- Q3 Structured Server Log Error Analyser 25 marks
- Q4 Email Domain Frequency Analyser 25 marks

4/4 answered

Q4 Email Domain Frequency Analyser

25 marks

Context: An organization maintains a large dataset of employee email addresses stored as strings with varying formats. The system must extract domain names from each email entry using string splitting. It should count how frequently each domain appears in the dataset. The most commonly used domain must be identified and reported. Control flow constructs must guide counting and comparison logic. Functions should be implemented for extraction and analysis processes. The program must handle invalid or malformed email addresses gracefully.

Task:

Design a Python solution for comprehensive email domain frequency analysis.

Key Skills to Demonstrate:

- String splitting using split('@')
- Domain extraction and storage
- Frequency counting using loops and conditionals
- Identifying maximum frequency
- Handling malformed emails using conditionals

Your Code

```
def analyse(emails):
    count={}
    for email in emails:
        if "@" in email:
            domain=email.split("@")[1]
            if domain in count:
                count[domain]+=1
            else:
                count[domain]=1
        else:
            print("invaild email:",email)
    print("domain frequency:")
    for domain in count:
        print(domain,":",count[domain])
    maxdomain=""
```

Input

```
4
abc@gamil.com
123@yahoo.com
1233@saveetha.com
```

Output

```
domain frequency:
gamil.com : 1
yahoo.com : 1
saveetha.com : 2
most common domain: saveetha.com
```

SIMATS Online Lab

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=389

Ask Gemini | Verify that it's you | All Bookmarks

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

H Harshavardhan 192511177IRC

Q4
Email Domain Frequency Analyser
25 marks

4/4 answered

- Frequency counting using loops and conditionals
- Identifying maximum frequency
- Handling malformed emails using conditionals

Your Code

```
def analyse(emails):
    count={}
    for email in emails:
        if "@" in email:
            domain=email.split("@")[1]
            if domain in count:
                count[domain]+=1
            else:
                count[domain]=1
        else:
            print("invaild email:",email)
    print("domain frequency:")
    for domain in count:
        print(domain,":",count[domain])
    maxdomain=""
    maxcount=0
    for domain in count:
        if count[domain]>maxcount:
            maxcount=count[domain]
            maxdomain=domain
    if maxdomain!="":
        print("most common domain:",maxdomain)
        print("frequency:",maxcount)
n=int(input())
emails=[]
for i in range(n):
    emails.append(input())
analyse(emails)
```

Input

```
4
abc@gamil.com
123@yahoo.com
1233@saveetha.com
```

Output

```
domain frequency:
gamil.com : 1
yahoo.com : 1
saveetha.com : 2
most common domain: saveetha.com
```

Submitted